Bounded-State Categorical Voting for Online Sequence Learning

A White-Box Alternative to Length-Growing Black-Box Models

Anonymous Author

March 2026

Abstract

We propose a different starting point for sequence modeling: if both perceptual inputs and behavioral outputs are ultimately handled as categorical responses, then a sequence model need not reconstruct continuous history in full; it only needs to preserve enough bounded evidence to rank the correct categorical response above competing alternatives. From this viewpoint, memory is not literal retrieval but a context-triggered re-instantiation of categorical evidence. Based on this premise, we introduce a white-box architecture for online sequence learning built from five components: deterministic hash-based lexical coding, multi-timescale exponential moving average (EMA) state compression, a bounded positional feature map, a sparse short-term voting memory, and a low-rank long-term knowledge store trained by sleep-style consolidation. The resulting system supports continual online inference, continual online learning, and an inspectable decision path. After bounded-state compression, inference time and memory usage are constant with respect to processed sequence length. In early experiments, the system reaches 87.0% next-token accuracy in wake-only mode on `tinystories_11_20.txt`, 85.6% after transferring predictive content into the long-term module alone, 84–86% A-only and 92–93% A+B on `tinystories_10.txt`, 85% on the first 200 lines of PTB, 100% on a 230-word memory test, and shows controlled zero-shot lexical ecological generalization. We present this work as a research program: an interpretable, bounded-state alternative to black-box sequence models whose complexity grows with explicit context replay.

1 Introduction

The standard question in modern sequence modeling is how to preserve or retrieve more history. This question has produced increasingly sophisticated mechanisms for caching, attending, compressing, or retrieving past context. Yet it leaves one deeper assumption largely untouched: that accurate next-token prediction fundamentally requires access to a detailed replay of past sequence content.

This paper starts from a different assumption. We argue that once input and output behavior are viewed through the lens of categorical response, sequence learning can be reformulated. The model need not reconstruct continuous history in full detail. Instead, it must preserve enough bounded evidence for the correct categorical response to win a structured vote among alternatives. Put differently, no matter how complicated the internal mechanism becomes, the final act of next-token prediction is still a competition among discrete candidates. In that precise sense, even a modern large language model still ends in voting.

This shift is motivated by two ideas. First, perceptual and linguistic processing appears to depend on discretized internal responses to salient structure, rather than on literal storage of raw signal. Second, voting among categorical responses requires a very different level of precision from exact continuous reconstruction. That lower precision requirement opens the door to bounded state, sparse memory, and explicit online learning.

The architecture introduced here is built around this premise. It treats sequence modeling not as black-box function approximation over an ever-growing history, but as bounded-state categorical voting driven by explicit memories. The resulting system combines a deterministic lexical frontend, a multi-timescale state compressor, a bounded feature map, a fast short-term voting memory, and a slower long-term factorized memory that absorbs patterns through consolidation.

The claim of this paper is not that the present system has already surpassed large-scale language models on broad benchmarks. The claim is more specific: *if discrete response selection is treated as the first principle, then sequence learning admits a different architectural regime—one based on bounded state, voting-style memory, continual online updates, and strong interpretability.*

Contributions. This draft makes five contributions:

1. It proposes categorical response selection, rather than continuous history reconstruction, as the first principle for a new sequence-learning regime.
2. It formulates memory as context-triggered categorical re-instantiation rather than literal database-style retrieval.
3. It introduces a concrete white-box architecture whose inference and memory usage are constant with respect to processed sequence length after bounded-state compression.
4. It shows early evidence that a low-rank long-term module can absorb most of the predictive content of a fast short-term memory.
5. It outlines a broader research program around bounded-state sequence modeling, online learning, and explicit memory.

2 Notation and Implementation Conventions

Table 1 lists the symbols used in the implementation-aligned part of the paper.

3 Categorical Response as First Principle

The conceptual starting point of this work is simple: the model's interaction with the world is mediated through discrete, enumerable response spaces rather than through perfect reconstruction of continuous reality. Inputs may originate in complex physical signals, but the model only acts on features that trigger internal categorical responses. Outputs are also selections among discrete candidates.

From this viewpoint, sequence modeling changes character. The objective is no longer to preserve a continuously faithful reconstruction of everything that happened before. The objective is to preserve enough state for the right category to be selected when the next response must be made. This is also why the present work parts ways with the dominant long-context framing: if the end task is categorical selection, then the central problem is not exhaustive retrieval of history, but preservation of decision-relevant pressure on the current vote.

This leads to three consequences.

- **Only categorical systems can vote.** Voting is meaningful only when the response space is discrete.

Symbol	Shape	Meaning in the current implementation
K	scalar	Slot-space size used by `HashRetina`
N	scalar	Number of EMA timescales
D	scalar	Feature dimension used by the positional encoder and by stored B vectors
d	scalar	Rank of the long-term factorization `CP_RANK`
c_w	sparse map	Slot-count dictionary returned by `HashRetina.encode(w)`
Q_i	$\mathbb{R}^N$	EMA state row associated with slot i
ϕ_i	$\mathbb{R}^D$	Feature vector produced for active slot i
$\widehat{\phi}_i$	$\mathbb{R}^D$	Rowwise normalized feature vector used in scoring and wake updates
$b_{s,t}$	$\mathbb{R}^D$	Sparse short-term edge vector stored for source-target slot pair (s,t)
$E_{\text{src}}[s]$	$\mathbb{R}^{d\times D}$	Long-term source factor for slot s
$E_{\text{tgt}}[t]$	$\mathbb{R}^{d\times D}$	Long-term target factor for slot t
$A[s,t]$	$\mathbb{R}^D$	Reconstructed long-term feature vector for edge (s,t)
$\ell_B(t)$	scalar	Slot score contributed by short-term memory
$\ell_A(t)$	scalar	Slot score contributed by long-term memory

Table 1: Symbols and shapes used in the implementation-aligned formulation.

- **Voting tolerates compression.** To rank the correct response above competitors, the system needs sufficient evidence, not exact replay of past signal.
- **Memory becomes trigger-based.** What matters is not exact retrieval of stored history, but the re-activation of the right categorical evidence under the right state.

We therefore adopt the following thesis:

> Because input and output spaces are categorical and enumerable, sequence modeling need not reconstruct continuous history; it only needs to preserve enough bounded evidence for correct categorical voting.

4 Memory Is Trigger, Not Retrieval

A central consequence of the previous section is a new interpretation of memory. In this work, memory is not treated as a database process. It is not defined by exact read/write access to a fully stored past. Instead, memory is treated as *context-triggered categorical reconstruction*: under the right state, the system re-instantiates enough evidence for the relevant category to emerge.

In informal terms, memory is not retrieval but a triggered illusion under the right state. This wording is intentionally provocative, but the technical point is precise: the system does not need to recover the past itself; it needs to recover the correct pressure on the current categorical decision.

This idea also explains why bounded temporal encoding matters. A bounded-state temporal code such as multi-timescale EMA allows the model to represent the activation history of a category without expanding explicit time structure. The system does not store time as a replay buffer. It stores time as state. In the current implementation, the immediate trigger variable is the Q state itself: the active rows of Q encode which categories are currently alive, at what relative strengths,

and across which decay horizons. If the present Q state aligns with previously learned source conditions, then the corresponding evidence is re-instantiated by inner products in B and A.

5 Why Bounded State Becomes Possible

If memory were literal retrieval, then compression would face a hard obstacle: exact recall capacity would scale directly with stored detail. In that regime, finite parameters would eventually fail to carry growing history. This is one reason the present work rejects precise retrieval as the first principle.

By contrast, once the goal is recast as categorical voting, the precision requirement changes dramatically. Local evidence need only shift the correct candidate above competing alternatives. This makes bounded compression plausible. The model does not need to remember everything; it needs to preserve whatever is still decision-relevant.

We therefore treat thinking as strongly autoregressive and incrementally recursive: each step depends on the current local state plus a compressed carry-over of the past, rather than on instantaneous replay of the entire history.

6 Implemented Architecture

This section defines the implementation actually used in the current code path. We restrict attention to the default research setting used by the current experiments: cosine-chaos positional encoding, energy-style wake updates, and MLX-based sleep consolidation.

Let K denote the slot-space size, N the number of EMA timescales, D the feature dimension used by the positional encoder, and d the CP-style rank used by the long-term memory. In the current implementation, D equals `cfg.d`; for chaos mode this is `D_PHI`, while the long-term rank is `CP_RANK`.

6.1 HashRetina: token to sparse slot counts

For a token w, the frontend constructs the padded string `^w$` and extracts character n-grams for each scale in a finite set $\mathcal{N}$ (default: $\mathcal{N} = \{3\}$). Each n-gram is mapped by a deterministic hash into a slot in $\{0, \ldots, K-1\}$. The encoder returns a finite count dictionary

$$c_w : \{0, \ldots, K-1\} \to \mathbb{N}, \tag{1}$$

where $c_w(j)$ is the total multiplicity of hashed n-grams landing in slot j. In the implementation, different n-gram scales are separated by prefixing the hash input with the scale index.

This frontend is not a semantic embedding layer. Its main role is to produce a bounded sparse slot representation while allowing partial slot sharing across morphologically related words.

6.2 Multi-timescale EMA state

The context state is stored as a sparse set of active slot rows with values in $\mathbb{R}^N$. Let $Q_i(t) \in \mathbb{R}^N$ denote the row associated with slot i at step t, and let

$$\rho = (\rho_1, \ldots, \rho_N) \in (0,1)^N \tag{2}$$

be the per-timescale retention vector.

The implemented update is not written as a single closed form in the code. Operationally it is:

1. decay every currently active row elementwise,

$$Q_i(t) \leftarrow Q_i(t) \odot \rho; \tag{3}$$

2. remove rows whose maximum magnitude falls below a small threshold;

3. for every slot j present in the current count dictionary c_w, add the scalar count $c_w(j)$ to *all* timescales of that row,

$$Q_j(t+1) \leftarrow Q_j(t) + c_w(j)\mathbf{1}_N. \tag{4}$$

Thus the current token contributes equally to all timescales at insertion time; timescale separation arises through subsequent decay. Active rows are those whose row norm exceeds the threshold used in the implementation.

6.3 EMA thresholding and active-slot saturation

The bounded-state interpretation of EMA can be stated briefly. Fix a slot space of size K, a decay vector $\rho \in (0,1)^N$, and an activation threshold $\theta_Q > 0$. Suppose the per-step inserted count into any slot is bounded above by $c_{\max}$. Then every coordinate of every EMA row is bounded by its geometric-series envelope, namely

$$Q_i^{(n)}(t) \leq \frac{c_{\max}}{1-\rho_n}. \tag{5}$$

Hence the online state cannot grow with raw history length in an unbounded way. After thresholding, the number of active rows is bounded by the finite slot budget and threshold rule; in particular, it is independent of processed sequence length. The stronger empirical phenomenon observed in this project is *active-slot saturation*: beyond a sufficiently long context, the number of rows surviving thresholding stabilizes in a bounded range determined by slot budget, decay rates, and thresholding, rather than by raw history length itself. This is the exact sense in which EMA turns temporal extent into bounded state.

6.4 Cosine-chaos feature map

The standard positional feature map in the present paper is the cosine-chaos encoder. For a matrix of active EMA rows $Q_{\text{act}} \in \mathbb{R}^{m\times N}$, define

$$\widetilde{Q} = Q_{\text{act}} \odot \alpha_{\text{scale}}, \tag{6}$$

where $\alpha_{\text{scale}} \in \mathbb{R}^N$ is the inverse-weight vector used in the implementation. Let $P \in \mathbb{R}^{D\times N}$ be the fixed random projection matrix. The initial projection is

$$E = P\widetilde{Q}^\top \in \mathbb{R}^{D\times m}. \tag{7}$$

The recurrence starts from $M_0 = 0 \in \mathbb{R}^{D\times m}$ and iterates

$$M_t = \cos\big(\alpha(\text{shift}(M_{t-1}) + E)\big), \qquad t = 1, \ldots, L, \tag{8}$$

where $\text{shift}(\cdot)$ is the cyclic shift along the feature axis, α is the chaos parameter, and L is the number of folds. The final feature matrix is

$$\phi = M_L^\top \in \mathbb{R}^{m\times D}. \tag{9}$$

In the current default setting, $\alpha = 2$, motivated by the Lyapunov calculation

$$\lambda = \log(\alpha/2), \tag{10}$$

which places $\alpha = 2$ at the edge of chaos under the mean-field approximation used in the accompanying analysis notes. This analysis motivates the default parameter choice, but the implemented feature map is exactly the recurrence above.

6.5 Short-term memory B as a sparse edge map

The short-term memory is implemented as a sparse hash table over ordered slot pairs. Each stored edge has the form

$$B[(s,t)] = b_{s,t} \in \mathbb{R}^D, \tag{11}$$

where s is a source slot, t is a target slot, and $b_{s,t}$ is a feature-space vector. The implementation also stores a last-update time for each edge and applies lazy decay at access time.

This means that B is conceptually a sparse collection of exact voting traces, but mathematically in the code it is not represented as a dense tensor. It is a finite edge map from slot pairs to D-dimensional vectors. A useful intuition is the transpose view of the same object. In a dense formulation one may imagine a tensor of size $V_{\text{tgt}} \times (V_{\text{src}} \times D)$, where each target stores the source-conditioned feature pattern that supports it. Reading the same structure transversely gives $V_{\text{src}} \times (D \times V_{\text{tgt}})$: for a currently active source slot and its feature vector, one asks which target conditions are most compatible. The implemented sparse edge map realizes exactly this source-conditioned voting view without materializing the dense tensor.

6.6 Long-term memory A as slice-wise factorization

The long-term memory stores two dictionaries of slot-indexed matrices,

$$E_{\text{src}}[s], E_{\text{tgt}}[t] \in \mathbb{R}^{d \times D}. \tag{12}$$

For a source-target pair (s,t), the corresponding feature vector reconstructed by A is

$$A[s,t] = \sum_{r=1}^{d} E_{\text{tgt}}[t]_{r,:} \odot E_{\text{src}}[s]_{r,:} \in \mathbb{R}^D. \tag{13}$$

This is the precise object returned by the single-edge query routine in the current implementation.

7 Exact-Trace B and Embedded A

The distinction between B and A is not merely computational; it is the core epistemic division of the model.

B as exact trace. The short-term memory stores precise episodic voting traces. It is tied to discrete observed co-activations and therefore supports exact but weakly generalized behavior.

A as generalized structure. The long-term module replaces explicit trace storage with a low-rank embedding structure. In doing so, it discovers regularities across slots, clusters similar trace patterns, and supports generalization unavailable to purely exact retrieval.

This is the core reason that A can eventually take over most of B's predictive function: the system is not just copying traces, but factorizing them into a more structured space.

Algorithm 1 One-step inference and local wake update

Encode current token into sparse slot counts c_w with `HashRetina`
Decay active rows of Q, compact inactive rows, and insert c_w
Select active slots $\{s_i\}$ from Q and compute ϕ_i, then normalize to $\widehat{\phi}_i$
for all candidate target slots t touched by sparse support **do**
    Compute $\ell_B(t) = \sum_i \langle b_{s_i,t}, \widehat{\phi}_i \rangle$
    Compute $\ell_A(t) = \langle E_{\text{tgt}}[t], \sum_i E_{\text{src}}[s_i] \odot \widehat{\phi}_i \rangle_F$
end for
Predict the next response by ranking targets with $\ell(t) = \ell_A(t) + \ell_B(t)$
for all true next-token target slots t **do**
    Write to B with $\Delta b_{s_i,t} = \eta \left[1 - (\gamma s_A + s_B)\right] \widehat{\phi}_i$
end for

8 Prediction as Voting

Given a token context, the system proceeds as follows:

1. encode each token into slots,
2. update the bounded EMA state Q,
3. extract active slots and compute bounded feature vectors ϕ,
4. score target slots through both B and A,
5. decode slot scores into lexical predictions.

Let $\phi_i \in \mathbb{R}^D$ denote the feature vector associated with active slot s_i, and let

$$\widehat{\phi}_i = \frac{\phi_i}{\|\phi_i\|_2 + 10^{-8}}. \tag{14}$$

The implementation normalizes feature vectors before both A-strength and B-strength computations. When the same source slot is paired with multiple target slots during wake, the normalized source feature is repeated across those target pairs.

At slot level,

$$\ell(j) = \ell_B(j) + \ell_A(j). \tag{15}$$

The long-term score is computed by first aggregating a source-conditioned context tensor

$$C_A = \sum_i E_{\text{src}}[s_i] \odot \widehat{\phi}_i \in \mathbb{R}^{d \times D}, \tag{16}$$

then scoring every target slot t by Frobenius inner product,

$$\ell_A(t) = \langle E_{\text{tgt}}[t], C_A \rangle_F. \tag{17}$$

The short-term score is computed from the stored edge vectors:

$$\ell_B(t) = \sum_i \langle b_{s_i,t}, \widehat{\phi}_i \rangle, \tag{18}$$

where the sum only ranges over edges actually present in the sparse edge map. Prediction is therefore the merge of episodic evidence and generalized evidence. The operative point is that the model does not retrieve a stored sentence or a stored event. It accumulates local votes from those source-conditioned traces whose trigger conditions are currently active in Q.

9 Wake–Sleep Learning

9.1 Wake: rapid local updates

Wake learning writes rapidly into B using only local information. For each active source slot s_i and each target slot t of the next token, the implementation computes

$$s_B(s_i, t) = \langle b_{s_i,t}, \widehat{\phi}_i \rangle, \tag{19}$$

with $s_B = 0$ if the edge is absent, and

$$s_A(s_i, t) = \Big\langle \mathrm{vec}\big(E_{\mathrm{src}}[s_i] \odot \widehat{\phi}_i\big),\ \mathrm{vec}\big(E_{\mathrm{tgt}}[t]\big) \Big\rangle, \tag{20}$$

where $\widehat{\phi}_i$ is broadcast across the first axis of $E_{\mathrm{src}}[s_i]$. This is exactly the flattened matrix product used by the current cached implementation of A-strength during wake. The implemented energy gap is then

$$g(s_i, t) = 1 - (\gamma\, s_A(s_i, t) + s_B(s_i, t)), \tag{21}$$

with $\gamma =$ `GAMMA_DECAY` in the current code path. The wake update writes

$$\Delta b_{s_i,t} = \eta\, g(s_i, t)\, \widehat{\phi}_i. \tag{22}$$

This is the default update used by the current experiments. Its purpose is to prevent high-frequency patterns from growing without bound once A and B already provide sufficient support. Crucially, wake learning is online and does not require full backpropagation-through-time.

9.2 Sleep: compression of traces into structure

Sleep gathers the accumulated B entries and trains A to approximate targets of the form

$$\mathrm{target}_{ij} = \gamma A_{\mathrm{old}}[i, j] + B[i, j], \tag{23}$$

or an adaptive-γ variant when slot-wise decay is enabled. More precisely, if the sampled wake edges are (s_k, t_k, b_k), then the old long-term prediction is

$$a_k^{\mathrm{old}} = A[s_k, t_k] \in \mathbb{R}^D, \tag{24}$$

and the sleep target is

$$y_k = \gamma_k a_k^{\mathrm{old}} + b_k, \tag{25}$$

with either a global $\gamma_k = \gamma$ or an adaptive slot-dependent coefficient in the current implementation.

The trainable parameters during sleep are stored internally as arrays of shape (D, d) per slot for computational reasons, but are converted back to the public (d, D) representation after optimization. The new-sample loss is

$$\mathcal{L}_{\mathrm{new}} = \frac{1}{n_{\mathrm{new}}} \sum_k \ell(A_\theta(s_k, t_k) - y_k), \tag{26}$$

where ℓ is either elementwise MSE or Huber loss in the implementation. When rehearsal is enabled, additional pairs are sampled from already-known slots and penalized toward decayed old predictions:

$$\mathcal{L}_{\mathrm{reh}} = \frac{1}{n_{\mathrm{reh}}} \sum_u \ell(A_\theta(\tilde{s}_u, \tilde{t}_u) - \tilde{y}_u). \tag{27}$$

The full sleep objective is

$$\mathcal{L} = \mathcal{L}_{\text{new}} + \mathcal{L}_{\text{reh}}, \tag{28}$$

optimized with AdamW in the current MLX implementation. Thus, optimization appears in the architecture as a consolidation tool rather than as the general learning principle of the whole system.

The philosophical meaning of sleep is simple: exact traces in B are distilled into generalized structure in A.

10 Complexity Claim

We state the complexity claim carefully.

Sequence-length complexity. Once history has been compressed into bounded EMA state, inference time and memory usage are *constant with respect to total processed sequence length.* The system therefore avoids the growing cost of explicit context replay.

Why the claim is well-defined. The $O(1)$ claim is made with respect to already-processed sequence length, not with respect to every quantity in the model. Its key structural assumption is that EMA-based temporal encoding yields active-slot saturation under a fixed activity threshold: beyond a sufficiently long context, the number of active rows that survive thresholding is bounded by the slot budget and threshold rule rather than by raw history length. Under this bounded-active-support regime, the online path operates at state-model scale: it carries a fixed-dimensional state, computes fixed-dimensional features, and reads or writes only sparse slot-conditioned traces. In practice there remains a nonzero constant base cost from hashing, thresholding, sparse bookkeeping, and candidate decoding, but that cost does not grow with processed context length.

Training complexity. Wake training is linear in the number of observed positions, with bounded local computation per position under sparse active support. The architecture does not require backpropagation-through-time, so the online training memory footprint is constant with respect to sequence length. Feature-generation terms tied to the positional encoder can be precomputed or cached, so a conservative statement is linear time in the number of observed positions with constant sequence-length memory. Sleep training is separate and depends on the number of stored B entries and the rank of the long-term factorization.

Parallel wake aggregation. Because wake writes are local edge updates, they can also be accumulated in parallel over shards and merged afterward by weighted summation on coincident slot pairs. This observation is not yet the center of the present experiments, but it is a genuine architectural property of the implemented sparse update rule.

Scope of the claim. This is not a claim that every aspect of computation is constant in vocabulary size, slot count, or total stored memory size. It is a claim that the architecture removes dependence on raw context length by compressing history into bounded state.

Implementation alignment. For mathematical precision, the complexity claim in this paper refers to the implemented bounded-state path: hash-based token coding, sparse EMA state, cosine-chaos feature generation in fixed dimension, sparse edge-map B, and factorized long-term memory A. It does not claim constant dependence on all other quantities in the implementation. What is

claimed strictly is constant dependence on processed sequence length after bounded-state compression, together with constant online training memory due to the absence of BPTT.

Audit note. Two implementation details matter for mathematical accuracy. First, the EMA insertion step adds the current slot count equally to all timescales and relies on subsequent decay to separate timescales. Second, the wake update uses normalized feature vectors $\widehat{\phi}$ rather than raw ϕ. Both points are reflected explicitly in the equations above because they materially affect the implemented dynamics.

11 Interpretability

The model is inspectable at every level:

- lexical slot activations from hash coding,
- multi-timescale EMA state Q,
- bounded voting features ϕ,
- explicit short-term voting traces in B,
- low-rank generalized factors in A.

This is not post hoc explanation. The decision path is part of the architecture itself.

12 Sparsity and Parameter Efficiency

The architecture is sparse by construction. The frontend produces sparse slot counts, thresholding keeps only active rows of Q, wake writes only to touched source-target pairs, and A replaces dense pairwise storage with a low-rank factorization over slots. As a result, most potential connections are never materialized.

This sparsity is not merely an implementation trick. It is part of the mathematical regime. Because prediction is formulated as categorical voting under bounded active support, the model only needs to preserve the edges and factors that carry decision-relevant pressure. In that sense, parameter efficiency follows from the problem reformulation itself: the system does not spend capacity on reconstructing full history, only on maintaining sparse evidence and compressible structure.

13 Biological Correspondence

The system was not designed as a literal imitation of biology. The design process was mathematical first. However, the resulting structure aligns naturally with several biological themes: fast and slow learning systems, bounded decaying context traces, continual online processing, and categorical competition among candidate responses. We therefore treat biological correspondence as corroborating evidence, not as the primary source of justification. The claim is not that the present equations prove neural identity with the brain. The more modest claim is that, once one follows the categorical-voting logic to its conclusion, human cognition itself begins to look compatible with this regime: memory appears less like database retrieval and more like context-sensitive re-evocation under bounded ongoing state.

Setting	Evaluation	Result
`tinystories_10.txt`	B-only next-token accuracy	87.0%
`tinystories_10.txt`	A-only after 10 cycles	84–86%
`tinystories_10.txt`	A+B combined	92–93%
`tinystories_11_20.txt`	Wake-only accuracy at cycle 20	87.0% (1342/1542)
`tinystories_11_20.txt`	A-only after sleep consolidation	85.6% (1320/1542)
PTB first 200 lines	early next-token experiment	85%
230-word memory test	memorization accuracy	100%
Controlled lexical ecology	zero-shot transfer (e.g., dog→wolf, not cat)	positive

Table 2: Early experimental results. Some are stable headline results; others remain best treated as internal results pending broader benchmark packaging.

14 Early Empirical Evidence

The current empirical package should be interpreted as evidence of viability rather than as a finished benchmark suite. Table 2 summarizes the most stable results currently suitable for discussion.

A takes over B. The most important empirical phenomenon is not the absolute score alone, but the fact that the long-term module can absorb most of the predictive content of the short-term module. On `tinystories_11_20.txt`, wake reaches 87.0% and the A-only model after sleep remains at 85.6%. On `tinystories_10.txt`, A-only reaches 84–86% while A+B reaches 92–93%. This indicates that sleep is not cosmetic: it transfers predictive structure into compressed long-term knowledge.

Controlled generalization. In controlled tests, an unseen lexical item occupying a similar ecological niche can inherit plausible next-token behavior through shared slot structure and context dynamics. For example, a context learned for *dog* may generalize to *wolf* but not to a less aligned token such as *cat*. We present this as early evidence of ecological generalization, not as a claim of broad semantic competence.

15 What This Challenges

The main target of this work is not any one architecture, but a prevailing assumption: that long-range sequence modeling is fundamentally a problem of preserving or retrieving as much history as possible. We challenge that assumption directly.

Our counter-claim is that once discrete response selection is treated as the first principle, the central problem changes. The model no longer needs exact history reconstruction. It needs bounded, decision-relevant evidence. This shift may enable a different family of systems with explicit memory, bounded state, and online learning as native properties.

16 Limitations

This draft represents an active research program, not a complete benchmark study. Several limitations remain:

- Results are currently strongest on synthetic and small-scale corpora.
- Baseline comparisons against standard sequence models are not yet systematized.
- Some claims, especially around scaling and efficiency, are more rigorously motivated than comprehensively benchmarked.
- The current empirical package is uneven: strong internal logs and repeated observations exist, but not all have yet been standardized into paper-ready benchmark tables.

17 Research Program

We see four immediate directions.

1. **Systematic benchmarking**: compare against n-gram, RNN/LSTM, transformer, linear-attention, and state-space baselines under matched context and memory budgets.
2. **Scaling laws**: characterize performance as a function of slot count, feature dimension, factor rank, and sparsity budget.
3. **Generalization diagnostics**: test ecological categories, syntax-like regularities, and compositional transfer beyond direct memorization.
4. **Hardware-efficient implementations**: exploit sparsity, hashing, and parallel operations to push the architecture toward edge deployment and continual-learning settings.

18 Conclusion

This work proposes a different starting point for sequence modeling: bounded-state categorical voting rather than explicit replay of continuous history. From this starting point, memory becomes trigger rather than retrieval, temporal structure becomes bounded state rather than expanding context, and learning becomes a division between rapid exact trace storage and slower structural consolidation. Early experiments suggest that this regime is viable. We do not claim that the current system is already a replacement for large-scale autoregressive language models. We do claim that it opens a technically concrete and intellectually distinct research direction that is both interesting and testable. If the central premise is right, then the payoff is larger than one architecture: it suggests that both machine sequence modeling and, perhaps in a compatible but not identical sense, brain-like cognition may be better understood as bounded categorical voting than as exact replay of growing history.

A Research Background

This project emerged from a broader line of inquiry into memory management for long-context language systems. One earlier engineering prototype, internally called *Bubble Stream*, used staged inference, memory block packing, and dynamic insertion into transformer context windows. In a

controlled setup, it achieved 94% memory precision over 32K cumulative dialogue using only 16K active context. That prototype is not the subject of this paper, but it helped motivate the present architecture by showing that memory behavior can depend more on memory-management structure than on raw context length alone.

B Status of the Evidence

The results in this draft are based on an actively iterated codebase with multiple documented design cycles. Some results are already stable enough for inclusion as headline evidence; others remain best treated as research observations pending standardized benchmark packaging. The purpose of the present draft is to communicate the architecture and research direction clearly enough for expert discussion, supervision, and collaborative development.